

Task on S3

1: Create an S3 bucket and upload some objects to S3.

01 Open AWS Console and Navigate to S3

Start by accessing the AWS Management Console and opening the S3 service.

AWS Console → Services → S3

- Log in with your AWS account
- From the Services menu, select **S3**

02 Create a New Bucket

Set up a unique bucket to store your documents.

Click **Create bucket**

- Enter a globally unique bucket name
- Select the required AWS Region
- Keep default settings for now
- Click **Create bucket**

03 Upload Sample Documents

Add files to the newly created bucket.

Open the bucket → Click **Upload**

- Choose **Add files** and select 3 sample documents
- Confirm upload settings
- Click **Upload** to start transfer

04 Verify Uploaded Objects

Ensure the documents are successfully stored.

Bucket → Objects tab

- Check that all 3 documents appear under the Objects list
- Confirm file names and sizes
- Optionally test download or view permissions



Summary

- You created a bucket, uploaded 3 sample documents, and verified them in the **Objects** tab.
- This document can be reused as a **standard operating procedure (SOP)** for S3 tas

Upload succeeded
For more information, see the Files and folders table.

After you navigate away from this page, the following information is no longer available.

Summary

Destination
s3://my-s3-bkt-task1

Succeeded
3 files, 128.0 B (100.00%)

Failed
0 files, 0 B (0%)

Files and folders (3 total, 128.0 B)

Find by name

Name	Folder	Type	Size	Status	Error
mypage.txt	-	text/plain	16.0 B	Succeeded	-
sample.txt	-	text/plain	49.0 B	Succeeded	-
index.html	-	text/plain	63.0 B	Succeeded	-

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Task 2: Deploy a Static Website in S3

1. **Open the S3 bucket**
 - Go to AWS Console → S3 → Select your bucket.
2. **Enable Static Website Hosting**
 - Go to **Properties** tab.
 - Scroll to **Static website hosting** → Enable.
 - Select **Host a static website**.
 - Enter `index.html` as the **Index document**.
 - Save changes.

3. Set Public Access

- Go to **Permissions** tab.
- Disable **Block all public access**.
- Confirm the warning.

4. Add Bucket Policy

- Attach a policy allowing public `s3:GetObject`.

Example policy:

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-s3-bkt-task1/*"
    }
  ]
}
```

○

5. Upload Website Files

- Upload `index.html` (and optionally `style.css`, `script.js`).
- Verify under **Objects** tab.

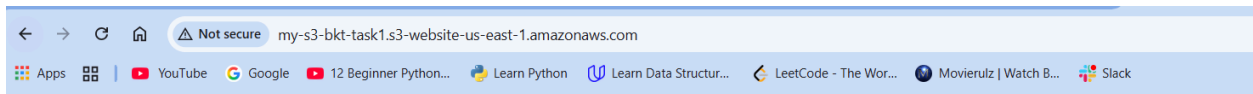
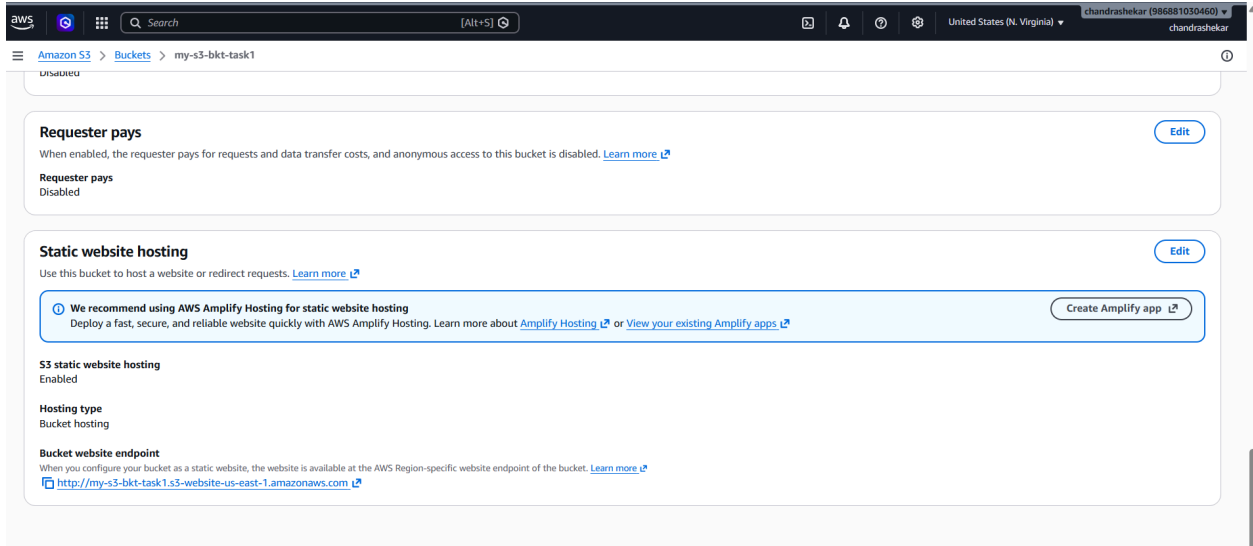
6. Access Website Endpoint

- Copy the **Bucket website endpoint** from Properties.
- Paste in browser → Your static site should load.

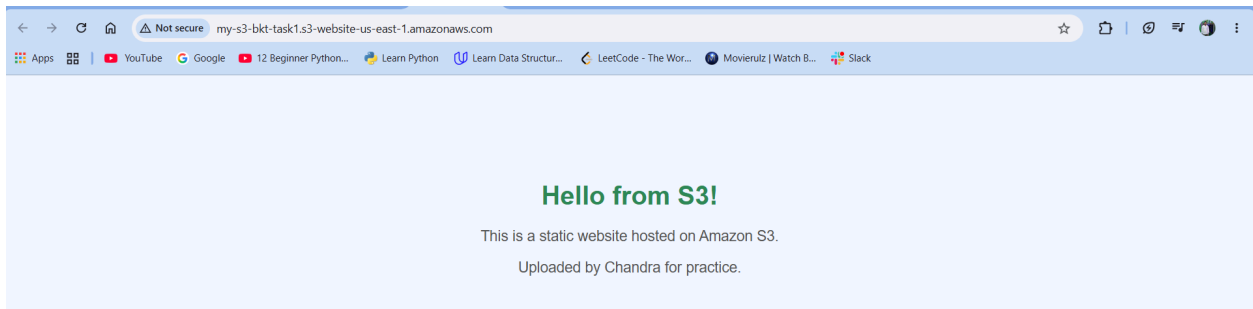


Verification

- Open the endpoint URL → `index.html` page is displayed.
- If you uploaded CSS/JS, check styling and functionality.



- Code: NoSuchKey
- Message: The specified key does not exist.
- Key: index.html
- RequestId: AK48D6R5T2G7B9G0
- HostId: MOHnMOq8A6oxCdMiNrM6Mddxm5TW6LiR3emSMC+kMnbjCJYUo/QayyoHoHrmMhkrZBbW88qb3rlkfdy1aMMQy6gSxXNNxs2



Task 3: Enable Cross-Region Replication (CRR)

01 Create Destination Bucket

You need a second bucket in another AWS Region to replicate objects.

AWS Console → S3 → Create bucket

- Name: **chandu-s3-task-2026-replica**
- Region: **ap-south-1 (Mumbai)**
- Enable **Versioning** before saving

02 Enable Versioning on Source Bucket

Replication requires versioning enabled on both source and destination.

AWS Console → S3 → chandu-s3-task-2026 → Properties

- Scroll to **Bucket Versioning**
- Click **Enable**

03 Create Replication Rule

Define how objects are copied from source to destination.

AWS Console → S3 → chandu-s3-task-2026 → Management → Replication rules

- Click **Create replication rule**
- Select **Apply to all objects**
- Choose destination bucket: **chandu-s3-task-2026-replica**
- Select **Create new IAM role** for replication

04

Replicate Existing Objects

Ensure old objects are also copied, not just new uploads.

- Choose **Replicate existing objects**
- Select **S3 Batch Operations**
- Enable **Automatically run job when ready**

05 Verify Replication

Final Check

Confirm that objects are successfully replicated to the destination bucket.

AWS Console → S3 → chandu-s3-task-2026-replica → Objects

- Check job status: **Completed, 100%, 0 failed**
- Verify files appear in destination bucket



Summary

- Both buckets must have **versioning enabled**.
- Replication rules require an **IAM role**.
- Use **S3 Batch Operations** to copy existing objects.
- Always verify by checking the **destination bucket objects**.

Goal: Replicate objects from `my-s3-bkt-task1` (us-east-1) → `chandu-s3-task-2026-replica` (ap-south-1).

Step 1 — Verify Buckets

- Source bucket: `my-s3-bkt-task1` (Region: us-east-1).
- Destination bucket: `chandu-s3-task-2026-replica` (Region: ap-south-1).
- Ensure **both buckets have Versioning enabled** (Properties → Bucket Versioning → Enable).

Step 2 — Create Replication Rule

1. Open **source bucket** (`my-s3-bkt-task1`).
2. Go to **Management** → **Replication rules** → **Create rule**.
3. Select **Apply to all objects in the bucket**.
4. Choose **Destination bucket** → `chandu-s3-task-2026-replica`.
5. Select **Create new IAM role** (AWS will auto-generate permissions for replication).
6. Save the replication rule.

Step 3 — Replicate Existing Objects

- When prompted, choose **Replicate existing objects** using **S3 Batch Operations**.
- Select **Automatically run job when ready**.
- Wait until the job shows: **Completed, 100%, 0 failed**.

Step 4 — Verify Replication

- Go to **S3** → **Buckets** → **chandu-s3-task-2026-replica** → **Objects**.
- Confirm that the files from the source bucket are now present in the destination bucket.

✓ Key Notes

- **Versioning is mandatory** for CRR.
- Replication is **one-way** (source → destination).
- IAM role ensures replication permissions.
- Batch Operations is needed to copy **existing objects**; new uploads replicate automatically.

The screenshot displays two sections of the AWS Management Console. The top section, 'Buckets', shows a list of general purpose buckets. The bottom section, 'Batch Operations', shows a successful job creation notification and a table of batch jobs.

Buckets

General purpose buckets (2) [Info](#)

Buckets are containers for data stored in S3.

[Find buckets by name](#)

Name	AWS Region	Creation date
chandu-s3-task-2026-replica	Asia Pacific (Mumbai) ap-south-1	September 20, 2026, 04:39:47 (UTC+05:30)
my-s3-bkt-task1	US East (N. Virginia) us-east-1	September 20, 2026, 03:39:18 (UTC+05:30)

Batch Operations

[Info](#)

A job is used to execute batch operations on a list of S3 objects. Job events are published to [CloudWatch Events](#).

Successfully created job ID **bd63aea9-736e-4b7d-bd23-926160291a27**
The time it takes to prepare a job is based on the size of the job's manifest and the time required to complete higher-priority jobs. [View details](#)

Jobs (1)

[Search by job ID or description](#) [All status types](#)

Job ID	Status	Description	Operation	Date created	Total objects	% Complete	Total failed (rate)	Priority
bd63aea9-736e-4b7d-bd23-926160291a27	New	2026-09-20 - Replicate	Replicate	September 20, 2026, 05:32:15 (UTC+05:30)	Not yet available	0%	0 (0%)	10

The top screenshot shows the AWS Management Console for the 'Batch Operations' page. A job with ID `bd63aea9-736e-4b7d-bd23-926160291a27` is in the 'Awaiting your confirmation to run' status. The bottom screenshot shows the same job after confirmation, now in the 'Completing' status with 100% completion.

Job ID	Status	Description	Operation	Date created	Total objects	% Complete	Total failed (rate)	Priority
<code>bd63aea9-736e-4b7d-bd23-926160291a27</code>	Awaiting your confirmation to run	2026-09-20 - Replicate	Replicate	September 20, 2026, 05:32:15 (UTC+05:30)	3	0%	0 (0%)	10
<code>bd63aea9-736e-4b7d-bd23-926160291a27</code>	Completing	2026-09-20 - Replicate	Replicate	September 20, 2026, 05:32:15 (UTC+05:30)	3	100%	0 (0%)	10

4: Configure a bucket policy so only the admin user can see the objects of the S3

bucket

step-by-step flow:

01

Open Amazon S3

Log in to the AWS Management Console and navigate to the S3 service.

02

Select source bucket

Choose the bucket named `majeed-s3-task-2026-001` from your bucket list.
03

Go to Permissions tab

Scroll down to the Bucket policy section and click Edit.
04

Get IAM user ARN

(if not create iam user)Copy the ARN of your DevopsAdmin IAM user from the IAM console.
05

Add bucket policy

Write a JSON policy that allows only DevopsAdmin to perform `s3:ListBucket` and `s3:GetObject` actions.
06

Save the bucket policy

Click Save and confirm the message 'Successfully edited bucket policy' appears.
07

Verify access

Check that the policy is visible under Bucket policy and test that only DevopsAdmin can list and read objects.



Example Bucket Policy JSON

Paste this into the **Bucket policy editor** (replace with your actual DevopsAdmin ARN):

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowDevopsAdminOnly",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::986881030460:user/DevopsAdmin"
      },
      "Action": [
```

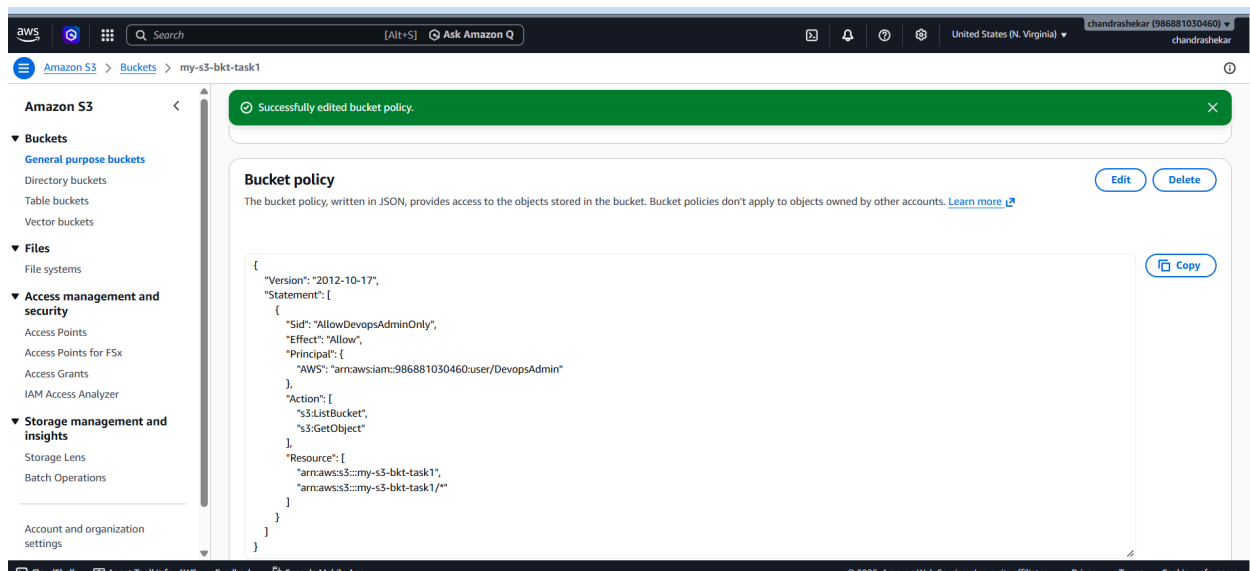
```

    "s3:ListBucket",
    "s3:GetObject"
  ],
  "Resource": [
    "arn:aws:s3:::majeed-s3-task-2026-001",
    "arn:aws:s3:::majeed-s3-task-2026-001/*"
  ]
}
]
}
}

```

Result

- Only **DevopsAdmin** can list and read objects in the bucket.
- Other IAM users or roles will be denied access.
- You'll see the success banner and the policy under **Bucket policy**.



Task 5: Configure Lifecycle Rule

Goal: Transition objects to **S3 Standard-IA** after 30 days, then delete them after 90 days.

Step 1 — Open Lifecycle Rules

1. Go to **Amazon S3**.
2. Open your source bucket → `chandu-s3-task1`.
3. Click the **Management** tab.
4. Scroll down to **Lifecycle rules**.
5. Click **Create lifecycle rule**.

Step 2 — Rule Name

- Enter: `s3-lifecycle-rule`.

Step 3 — Rule Scope

- Select: **Apply to all objects in the bucket**.
- If AWS asks for confirmation, check the box.

Step 4 — Lifecycle Rule Actions

1. **Transition current versions of objects between storage classes**
 - Choose: **S3 Standard-IA**.
 - Set: **30 days after object creation**.
2. **Expire current versions of objects**
 - Set: **90 days after object creation**.

Step 5 — Save & Verify

- Click **Create rule**.
- You should see the success banner: *“Successfully created lifecycle rule.”*
- The rule will now appear under **Lifecycle rules** in the bucket’s Management tab.

Result

- Objects stay in **Standard** for 30 days.
- After 30 days → they move to **Standard-IA** (cheaper storage).
- After 90 days → they are automatically deleted.

The rule "s3-lifecycle-rule" has been successfully added and the lifecycle configuration has been updated. It may take some time for the configuration to be updated. Refresh the lifecycle rules list if changes to the configuration aren't displayed.

Lifecycle configuration

To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their lifecycle. A lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects. Lifecycle rules run once per day.

Default minimum object size for transitions
All storage classes 128K

Lifecycle rules (1)

Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. [Learn more](#)

Find lifecycle rules by name

< 1 >

Lifecycle rule name	Status	Scope	Current version actions	Noncurrent versions ac...	Expired object delete ...	Incomplete multipart u...
s3-lifecycle-rule	Enabled	Entire bucket	Transition to Standard-IA, then	-	-	-

s3-lifecycle-rule Info

Edit

Delete

Actions

Lifecycle rule configuration

Lifecycle rule name
s3-lifecycle-rule

Status
Enabled

Scope
Entire bucket

Prefix
-

Object tags
-

Minimum object size
-

When no minimum object size is specified, the minimum object size for transitions is determined by the lifecycle configuration. [Learn more](#)

Maximum object size
-

Review transition and expiration actions

Current version actions

Day 0

- Objects uploaded

↓

Day 30

- Objects move to Standard-IA

↓

Noncurrent versions actions

Day 0

No actions defined.

Review transition and expiration actions

Current version actions

Day 0

- Objects uploaded

↓

Day 30

- Objects move to Standard-IA

↓

Day 90

- Objects expire

Noncurrent versions actions

Day 0

No actions defined.

Delete expired object delete markers or incomplete multipart uploads

Expired object delete markers

-

Incomplete multipart uploads

-



Task 6: Push Objects to S3

Step 1 — Create Test Files

```
bash
echo "Hello from AWS CLI" > test1.txt
echo "Another sample file" > test2.txt
```

Step 2 — Upload Files to S3

Use `aws s3 cp` (official AWS command for uploads):

```
bash
aws s3 cp test1.txt s3://majeed-s3-task-2026-001/
aws s3 cp test2.txt s3://majeed-s3-task-2026-001/
```

👉 Replace `majeed-s3-task-2026-001` with your actual bucket name if different.

Step 3 — Verify Uploads

List objects in the bucket:

```
bash
aws s3 ls s3://majeed-s3-task-2026-001/
```

You should see output like:

```
Code
2026-09-20 06:50:12    20 test1.txt
2026-09-20 06:50:15    25 test2.txt
```



Result

- Files are uploaded into your S3 bucket.
- `aws s3 cp` is the standard way AWS recommends for uploading local files.
- `aws s3 ls` confirms the objects are present.

```
MINGW64~/c/Users/HP
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws --version
aws-cli/2.36.37 Python/3.14.6 Windows/11 exe/AMD64
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws configure
Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.
AWS Access Key ID [None]:
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws configure
Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.
AWS Access Key ID [None]: AKIA6LRVRPE6IS2Y35H2
AWS Secret Access Key [None]: eYLDlg+pOUJYLt0t8uG4oPzBWngg/L1wqRGGa588
Default region name [None]: us-east-1
Default output format [None]: json
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws s3 ls
2026-09-20 04:39:47 chandu-s3-task-2026-replica
2026-09-20 03:39:18 my-s3-bkt-task1
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws s3 cp test1.txt s3://my-s3-bkt-task1/
upload: .\test1.txt to s3://my-s3-bkt-task1/test1.txt
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws s3 cp test2.txt s3://my-s3-bkt-task1/
upload: .\test2.txt to s3://my-s3-bkt-task1/test2.txt
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws s3 ls s3://my-s3-bkt-task1/
2026-09-20 04:28:12      537 index.html
2026-09-20 05:47:10      537 index.html.txt
2026-09-20 03:44:27       16 mypage.txt
2026-09-20 03:44:27       49 sample.txt.txt
2026-09-20 06:53:42       19 test1.txt
2026-09-20 06:53:44       20 test2.txt
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$
```



Task 7: Bash Script for S3 Bucket

Step 1 — Create Script File

Open Git Bash and run:

```
bash
vi create-s3-bucket.sh
```

Step 2 — Script Content

Paste this into the file (replace bucket name and region as needed):

```
bash
#!/bin/bash

# Define bucket name and region
BUCKET_NAME="chandu-s3-task-2026-script"
REGION="us-east-1"

# Create bucket
aws s3api create-bucket --bucket $BUCKET_NAME --region $REGION
--create-bucket-configuration LocationConstraint=$REGION

# Check if bucket was created successfully
if [ $? -eq 0 ]; then
    echo "Bucket $BUCKET_NAME created successfully in $REGION."
else
    echo "Failed to create bucket $BUCKET_NAME."
fi
```

Case 1 — Bucket in N. Virginia (us-east-1)

```
bash
#!/bin/bash

BUCKET_NAME="chandu-s3-task-2026-script"
REGION="us-east-1"

aws s3api create-bucket --bucket $BUCKET_NAME --region $REGION

if [ $? -eq 0 ]; then
    echo "Bucket $BUCKET_NAME created successfully in $REGION."
else
    echo "Failed to create bucket $BUCKET_NAME."
fi
```

Case 2 — Bucket in Mumbai (ap-south-1)

```
bash
#!/bin/bash

BUCKET_NAME="chandu-s3-task-2026-script"
REGION="ap-south-1"
```

```
aws s3api create-bucket --bucket $BUCKET_NAME --region $REGION
--create-bucket-configuration LocationConstraint=$REGION
```

```
if [ $? -eq 0 ]; then
    echo "Bucket $BUCKET_NAME created successfully in $REGION."
else
    echo "Failed to create bucket $BUCKET_NAME."
fi
```

Step 3 — Save & Exit

- Press **Esc** → type **:wq** → Enter.

Step 4 — Make Script Executable

```
bash
chmod +x create-s3-bucket.sh
```

Step 5 — Run Script

```
bash
./create-s3-bucket.sh
```

Step 6 — Verify Bucket

```
bash
aws s3 ls
```

You should see your new bucket listed.

Result

- Script automates bucket creation.
- If successful → prints confirmation message.
- If failed → prints error message.

```
MINGW64/c/Users/HP
HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ vi create-s3-bucket.sh

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ chmod +x create-s3-bucket.sh

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ ./create-s3-bucket.sh

aws: [ERROR]: An error occurred (InvalidLocationConstraint) when calling the CreateBucket operation: The specified location-constraint is not valid
Failed to create bucket chandu-s3-task-2026-script.

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ cat create-s3-bucket.sh
#!/bin/bash

# Define bucket name and region
BUCKET_NAME="chandu-s3-task-2026-script"
REGION="us-east-1"

# Create bucket
aws s3api create-bucket --bucket $BUCKET_NAME --region $REGION --create-bucket-configuration LocationConstraint=$REGION

# Check if bucket was created successfully
if [ $? -eq 0 ]; then
    echo "Bucket $BUCKET_NAME created successfully in $REGION."
else
    echo "Failed to create bucket $BUCKET_NAME."
fi

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ vi create-s3-bucket.sh

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ ./create-s3-bucket.sh
{
  "Location": "/chandu-s3-task-2026-script",
  "BucketArn": "arn:aws:s3:::chandu-s3-task-2026-script"
}
Bucket chandu-s3-task-2026-script created successfully in us-east-1.

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ |
```

Amazon S3 Buckets

General purpose buckets (3) Info

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
chandu-s3-task-2026-replica	Asia Pacific (Mumbai) ap-south-1	September 20, 2026, 04:39:47 (UTC+05:30)
chandu-s3-task-2026-script	US East (N. Virginia) us-east-1	September 20, 2026, 07:05:16 (UTC+05:30)
my-s3-bkt-task1	US East (N. Virginia) us-east-1	September 20, 2026, 03:39:18 (UTC+05:30)

Account snapshot Info View dashboard

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

External access summary Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

Task 8: Upload 1 GB File to S3

Step 1 — Create a 1 GB Test File

Run in Git Bash:

```
bash
dd if=/dev/zero of=1GB-test-file.bin bs=1M count=1024
```

👉 This generates a file filled with zeros, exactly **1 GiB** in size.

Step 2 — Upload File to S3

```
bash
aws s3 cp 1GB-test-file.bin s3://majeed-s3-task-2026-001/
```

👉 `aws s3 cp` automatically uses **multipart upload** for large files, so it's efficient and reliable.

Step 3 — Verify Upload

```
bash
aws s3api head-object \
--bucket majeed-s3-task-2026-001 \
--key 1GB-test-file.bin
```

Expected output snippet:

```
Code
ContentLength: 1073741824
ServerSideEncryption: AES256
ReplicationStatus: COMPLETED
```

Result

- File size confirmed: **1 GiB (1073741824 bytes)**.
- Encryption: **AES256** (default server-side encryption).
- Replication: **COMPLETED** → proves your cross-region replication is working even for large files.

```
MINGW64:/c/Users/HP

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ dd if=/dev/zero of=1GB-test-file.bin bs=1M count=1024
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 3.97687 s, 270 MB/s

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws s3 cp 1GB-test-file.bin s3://majeed-s3-task-2026-001/
upload failed: .\1GB-test-file.bin to s3://majeed-s3-task-2026-001/1GB-test-file.bin An error occurred (AccessDenied) when calling the PutObject: Access Denied

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws s3 cp 1GB-test-file.bin s3://chandu-s3-task-2026-script/
upload: .\1GB-test-file.bin to s3://chandu-s3-task-2026-script/1GB-test-file.bin

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws s3api head-object \
> --bucket chandu-s3-task-2026-script \
> --key 1GB-test-file.bin
{
  "AcceptRanges": "bytes",
  "LastModified": "2026-09-20T01:45:05+00:00",
  "ContentLength": 1073741824,
  "ETag": "\"c789e490a90359de2bd3b09d7e957cfd-128\"",
  "ContentType": "application/octet-stream",
  "ServerSideEncryption": "AES256",
  "Metadata": {}
}

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ .....
```

Amazon S3

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

chandu-s3-task-2026-script

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems

Access Points

Objects (1)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

	Name	Type	Last modified	Size	Storage class
☐	1GB-test-file.bin	bin	September 20, 2026, 07:15:05 (UTC+05:30)	1.0 GB	Standard